Fingerprint Recognition

Watch and check the theory video for this lab. You can also check the slides from the presentations in the material provided

System for automatic person recognition.

Each person has unique biometric characteristics that can be used to automatically determine the person's identity. Examples of such biometric characteristics are: fingerprints, face, iris, voice and hand geometry.

A system for automatic person recognition, called biometric system, typically consists of several modules, as shown in the figure:

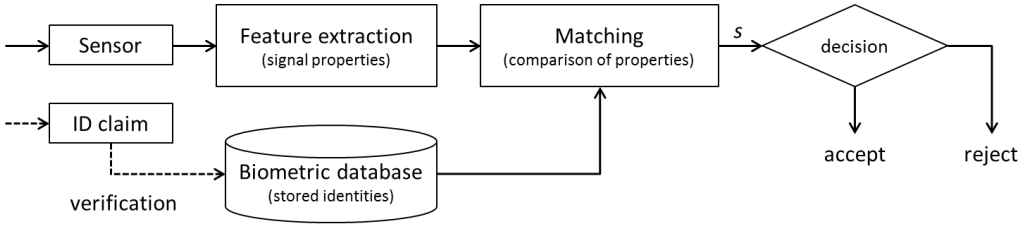

Figure 1. Biometric system. In identification mode (one-to-many comparison), only the sensor data is used to determine the person's identity. In verification mode (one-to-one comparison), the person must also claim a target identity. A decision regarding the person's identity is made based on the value of the similarity measure between the sensor data and the database data.

Fingerprints as biometric identifier.

When the fingerprint is used as biometric identifier, capacitive sensors are a common choice for imaging the fingerprint characteristics into an electronic image.

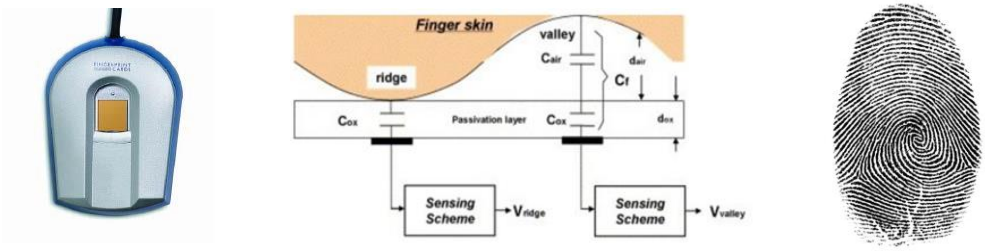

Figure 2. Left: fingerprint capacitive sensor. Center: imaging principle to obtain an electronic image with a capacitive sensor. Right: acquired fingerprint image.

A fingerprint capacitive sensor consists of a 2-dimensional array of micro-capacitor plates embedded in a chip. The finger itself constitutes the second plate to each micro-capacitor in the chip. The electrical charge of each capacitor plate varies depending on the topological pattern of a finger (composed of ridges and valleys). The charge stored in the capacitor will change when a finger's ridge is placed over the conductive plates, or when there is an air gap due to a valley.

Thanks to this principle, a (grayscale) electronic image of the fingerprint can be composed, see

Figure 2 (right). A grayscale image of the fingerprint consists of a matrix of numbers between 0 and 255 (the value is a measure of the charge of each micro-capacitor). The number 0 corresponds to black, and 255 to white, while the numbers in-between represent shades of grey between black and white (remember previous laboratory exercises where you represented and plotted images in Matlab using 2D matrixes!).

A common size of the sensor chip is about 0.7 x 0.5 inches (18 x 13 mm). With a resolution of 500 dpi, this means about 350 x 250 pixels (picture elements) in the grayscale image.

Detection of specific patterns of a fingerprint with linear filters.

A fingerprint of good quality consists of homogeneous dark and bright lines with different directions. Looking locally (in a small area around a pixel), most of the times the lines are parallel, i.e. they have the same direction. Only in certain points of the fingerprint image, the lines are not parallel. These points are called *singular points* and *minutia points* (we will call them S and M points). The position of these specific points is often used to compare two fingerprints, i.e. to check whether the two fingerprints come from the same finger or not. If two fingerprint images have many points in common in the same position, this increases the probability that the two fingerprint images under comparison are from the same finger (thus, from the same person).

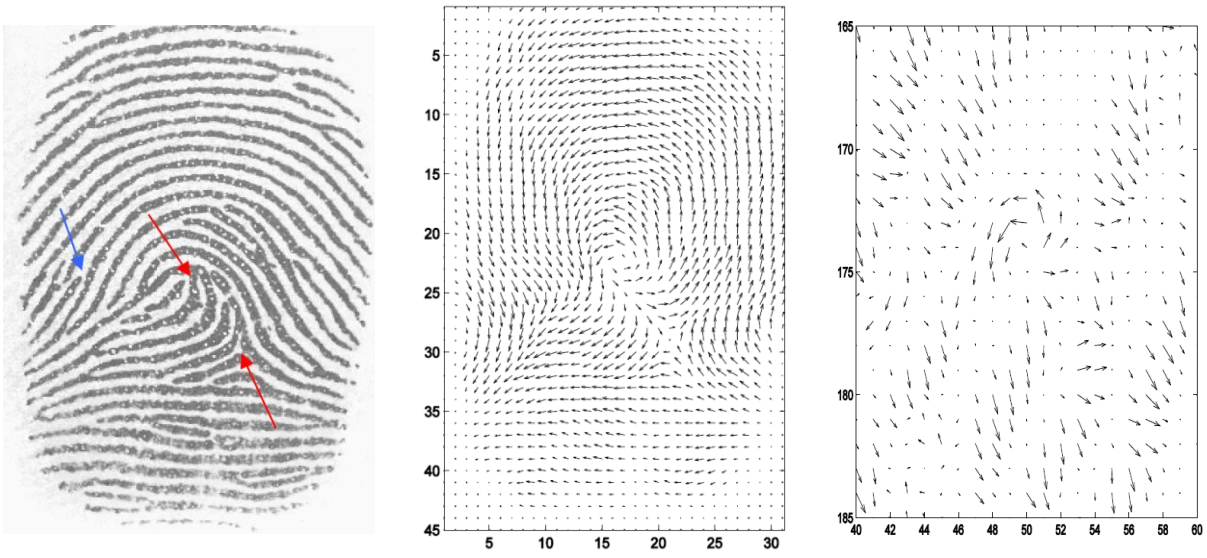

Figure 3. Left: grayscale image of a fingerprint. Center: local orientation around the two S points (red arrows). Right: local orientation around the M point (blue arrow). Double-angle-representation has been used in the representation (*see in page 2 of the document "4-fingerprints.pdf" why double angle representation is beneficia*l).

Detection of S and M points of a fingerprint can be made with filters that are designed to be sensitive to the particular variation in the directions that appear around these points. In other words, the filters are designed to give a strong response in the position of S or M points, and a small response (ideally zero) in other points of the image. The detection algorithm used in this exercise is based on this idea, and we will employ the filters of figure 4 for this purpose, which are designed to detect S and M points respectively.

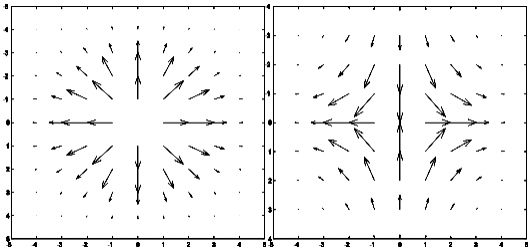

Figure 4. Filters designed to detect S and M points.

These filters are not applied to the grayscale image of the fingerprint, but to its orientation image (also called gradient image). *In the laboratory exercise 3 on Linear Filtering (Exercise 2.2: "Computing the gradient image"), for example, you calculated and displayed the gradient image of a face.* The orientation image is a vector image where each pixel is represented by two numbers (length and direction), compared with the grayscale image, where each pixel contains one number only (grey value). These vectors are calculated using the gradient of each pixel in the fingerprint image. The gradient is a vector whose **length** (magnitude) indicates how much the grayscale values changes locally (a higher length is obtained in sharp transitions from light-to-dark or dark-to-light, whereas a small length is obtained in areas where grayscale values does not change so much or are approximately constant). The **direction** of the gradient vector represents the direction in which the grey values change the most, i.e. perpendicular to the lines (in the algorithm, we use double angle representation, so the displayed orientation will not be perpendicular to the lines).

Linear filtering

In the previous laboratory exercise 3 you experimented with linear filtering of images. This section is a reminder of how it works.

Linear filtering of an image F (x, y) with a 2-dimensional filter w (x, y) is calculated by the convolution operation F * w. The convolution can be interpreted as follows: in each pixel (x, y) of the image, a new value g_0 is assigned based on the scalar product <f, w> between the filter w and a patch f of the image F around the pixel (x, y). The patch f must have the same size of the filter w.

A new pixel value g_0 is assigned at position (x, y) of the image, calculated as the scalar product between filter w and the patch f from the neighbourhood of pixel (x, y):

$$g_0 = \langle w, f \rangle = \sum_{i=0}^{9} w_i f_i = w_0 f_0 + w_1 f_1 + ... + w_8 f_8$$

where f_i (i= 0,1, ..., 8) are pixel values of the neighbourhood of pixel (x, y), and w_j (j= 0,1, ..., 8) are the filter coefficients (weights). See Figure 5, where a neighbourhood f is highlighted in red. Filtering the whole image means that the scalar product is calculated for each pixel of the image (an operation which is called **convolution**).

w_4	w_3	w_2
w_5	w_0	w_1
w_6	w_7	w_8

Figure 5. A new pixel value is calculated from the neighbourhood of the pixel as the scalar product <w, f> between the filter w and the image neighbourhood f. The filter size (here 3x3) and the values of the weights w_j determines the result of the filtering.

For example, if all the weights of a 3x3 filter have the value $w_j = 1/9$, we obtain an averaging filter (you can easily check that each pixel of the filtered image consists of the mean value of each 3x3 neighbourhood of the original image). If the weights are sampled from a Gaussian function, then we obtain a Gaussian, see Figure 6, which is another type of averaging filter (*remember laboratory exercise 3 on Linear Filtering).*

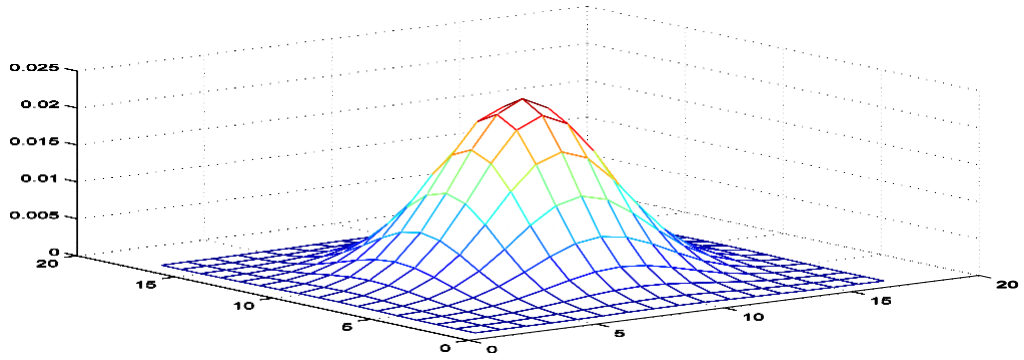

Figure 6. Gaussian filter.

<u>The exercise starts here.</u>

Download the compressed .rar file from Blackboard that is attached to this exercise.

<u>Exercise 1</u>
Run the detection algorithm *SingMpointExtr_2* on a fingerprint that has a spatially varying quality by entering the command **SingMpointExtr_2** in Matlab. Look at the intermediate results shown in the Matlab figures, and at the on-screen text.

This function takes as input the attached fingerprint image 41_1.tif and applies step by step the filters of figure 4 of this document to detect S and M points. Detection is done at different scales (levels) of the image, as explained in pages 5-6 of the attached document "4-

fingerprints.pdf" (section D on "Pyramid detection of core / m-points"). Read this section of the document to understand why and how detection is done at such different levels.

Remember that core points and m-points both have "parabolic" structure, so they will have a similar orientation pattern (look at figure 3 above). However, core points are "supported" by the entire image (all lines of the fingerprint are arranged around them), while m-points are smaller, so the orientation pattern of m-points will only be visible at low levels of the pyramid.

The result of the detection is shown in Figure 50 (S-points on the left, and M-points on the right).

Questions to answer:
a) How much width (in mm) has a ridge/valley in a human fingerprint?
Tip: the resolution of the sensor used to capture the fingerprint image of the exercise is 500 dpi, dpi = pixels per inch, and 1 inch = 25.4 mm, and you can appreciate the width in pixels of a ridge/valley from the grayscale image by making zoom.

b) How would you describe an area of poor quality of a fingerprint?
Tip: look at the magnitude of the vectors in the orientation image at level 1 or 2

c) Indicate the parts of the fingerprint that has poor quality

Exercise 2
Before detecting S and M points, we will now perform some enhancement to the grayscale image with a Gaussian filter (*remember from laboratory exercise 3 that Gaussian filters can be used for example to remove noise or in other words, to enhance an image*!). You can activate the enhancement in *SingMpointExtr_2* by setting the variable ENHANCE to 1 (lines 20/21, look at the labels ***CHANGE HERE***).

Make the change in the code using the Matlab editor, save and run the function. The result of the detection is shown in Figure 51.

This execution will recalculate all the intermediate figures. After examining them, you can close figures 1 to 9 for this exercise to remove clutter from the screen.

Questions to answer:

a) Was enhancement of the fingerprint successful?
Explain your answer by studying the results in Figure windows 50 and 51, which show the position of detected S and M points without and with enhancement respectively (for example, manually count the number of correct/incorrect detected M points in figure 50 and 51).

b) Find out and explain how the enhancement algorithm works.
You can get "help" by studying the enhancement algorithm more accurately executing the following Matlab commands:

```
>> DesignGaussFilter; % Construct 6 filters
>> figure(55);
>> clf(55); %clear figure 55
>> imo=double(imread('41_4.tif')); %read fingerprint image
>> imo=imo(3:362,:); %crop size to 360x256 pixels
>> imh=EnhFprGauss(imo); %filter fingerprint image
```

Figure 55 shows intermediate images used in the enhancement process.
Figure 56 and 57 show the filters used in the enhancement.

Examine the filtered images shown in Figure 55 and the enhanced image shown in Figure 49.
Hint: How many filtered images there are?
Which line direction is best seen in each filtered image?
Which filter has been used in the filtering of each image shown in Figure 55?

Exercise 3

In Figure 7, you can see how the 6 filtered images (Figure 55) are "weighted together" by linear interpolation to produce the enhanced image (Figure 49, right).

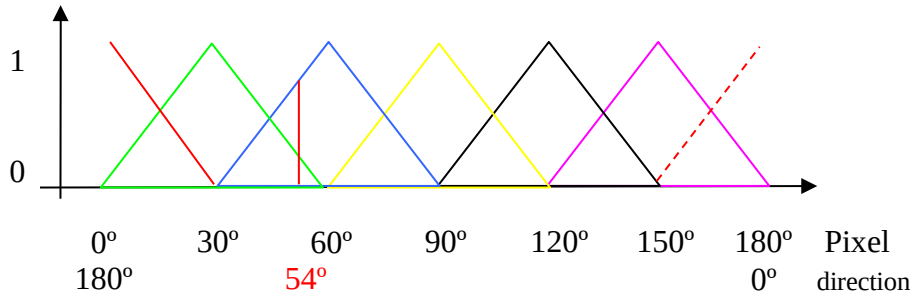

Figure 7. Linear interpolation of the 6 filtered images (represented by the colours: red (0º / 180º), green (30º), blue (60º), yellow (90º), black (120º) and purple (150º)) into a unique filtered picture. For example, pixel with an estimated direction φ = 54º is interpolated using the weights 0.2 and 0.8 to the corresponding pixel of images green and blue. The green image is the image filtered with a filter of angle 30º (= φ_{ref}) and the blue image with a filter of angle 60º (=φ_{ref}).

The local direction φ (x, y) of the lines for a pixel (x, y) is obtained from the orientation image. The pixel value f (x, y) of the enhanced fingerprint is then calculated from the two (out of 6) filtered images whose filter direction φ_{ref} is closest to φ. These filtered images are called "active". The weight of each "active" filtered image with the filter direction φ_{ref} is calculated as:

$$v\left(\varphi_{ref}\right) = 1 - \frac{\left|\varphi - \varphi_{ref}\right|}{30°} \quad \text{(Equation 3.1)}$$

Figure 7 shows an example with φ (x, y) = 54º. The two filtered images that are "active" are those with filter directions φ_{ref} = 30 ° (green) and φ_{ref} = 60 ° (blue). The weights are calculated as:

$$v(30°) = 1 - \frac{|54° - 30°|}{30°} = 0.2$$
$$v(60°) = 1 - \frac{|54° - 60°|}{30°} = 0.8$$

and the sum of the two weights is always = 1.

The pixel value f (x, y) is then calculated with the estimated weights according to:

$f(x,y) = 0.2 f_{30}(x,y) + 0.8 f_{60}(x,y)$ (Equation 3.2)

where $f_{30}(x,y)$ and $f_{60}(x,y)$ are values of pixel (x,y) in the filtered images with a direction of 30° (Figure 55, top row, middle) and 60° (Figure 55, top row, right).

Questions to answer:

To make sure that you understand how the 6 filtered images "weight together" by linear interpolation to build the enhanced fingerprint image, you will do the operation for at least two particular pixels of your choice at positions (r, c).

*You will use the included Matlab function LinjInt. Type **help LinjInt** to get information about this command. It is especially important that you understand the output parameters returned by LinjInt!*

Instructions: Attention!! please be aware that impixelinfo gives the coordinates as (x,y) instead of row and column (r,c). Be sure that you will be comparing the same point!!!!

1. *Point the mouse in the enhanced image (Figure 10) using the following code to obtain the position (r, c) and the grey scale value f (r,c). Annotate the position (r, c) and grey value f (r,c) at that position:*

   >> figure(10); imagesc(imh);colormap(gray);axis image; %display enhanced image

   >> impixelinfo;

2. *Calculate by hand f (r, c) with the value of the local orientation and grey values in the six filtered images that you get with the help of the command LinjInt. Use equations 3.1 and 3.2. Compare your grey value with the one you obtained when you "pointed" the mouse in figure 10. They should be the same!*

   >> load S; % load orientation images and filtered images

   >> r=?; % row index

   >> c=?; % column index

   >> out=LinjInt(r,c,S); %[φ,fim0,fim30,fim60,fim90,fim120,fim150]'

****

SUBMISSION OF THE REPORT.

Submit the report of this exercise no later than one week after the second lab session. Additional instructions are given in Blackboard.